

INTRODUÇÃO AO C/C++

Prof^a Gláucya Boechat

glaucya.boechat@ba.docente.senai.br

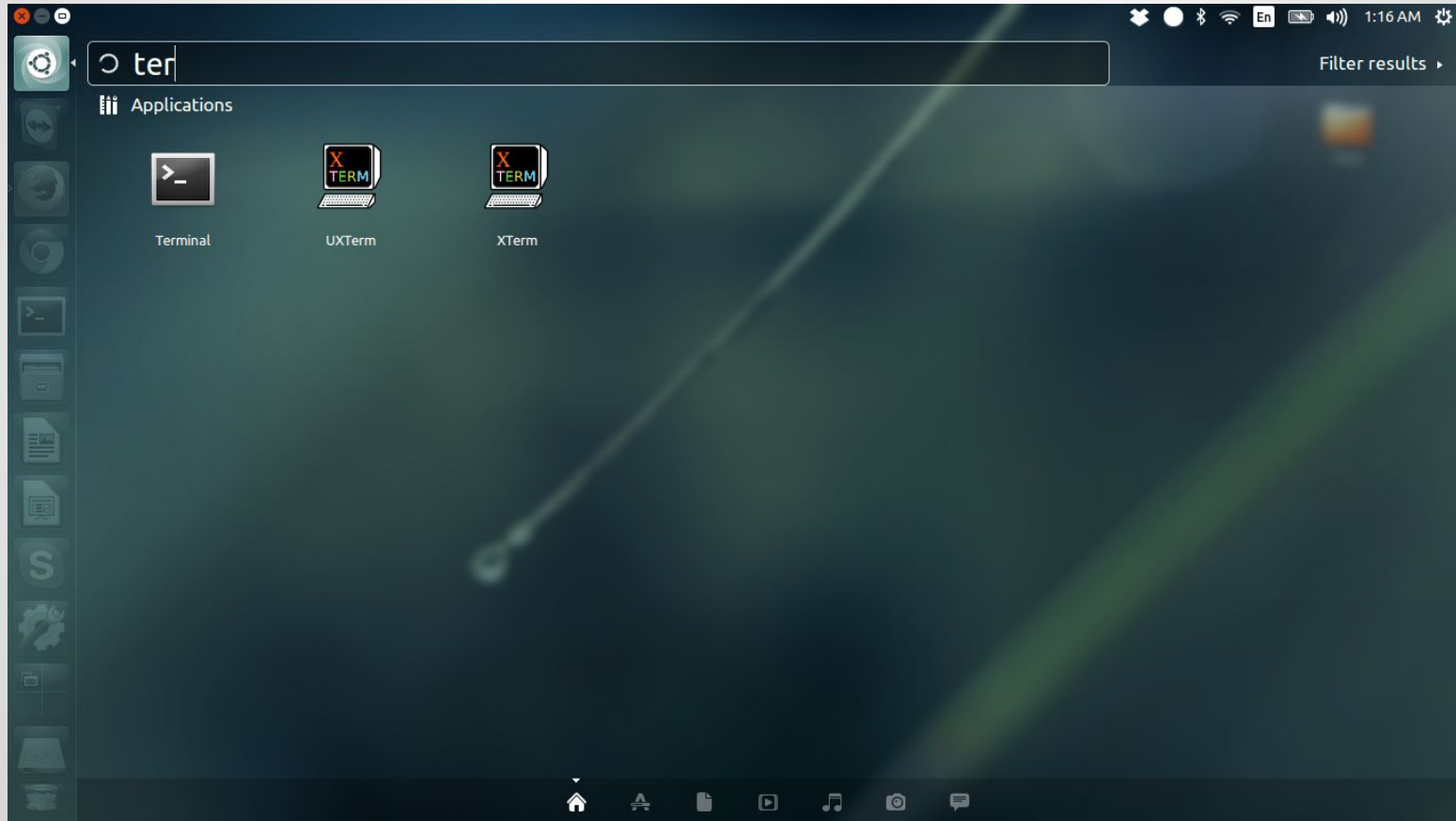
Slides cedidos pelo prof Rubisley de Paula Lemes
(IC - UFBA)



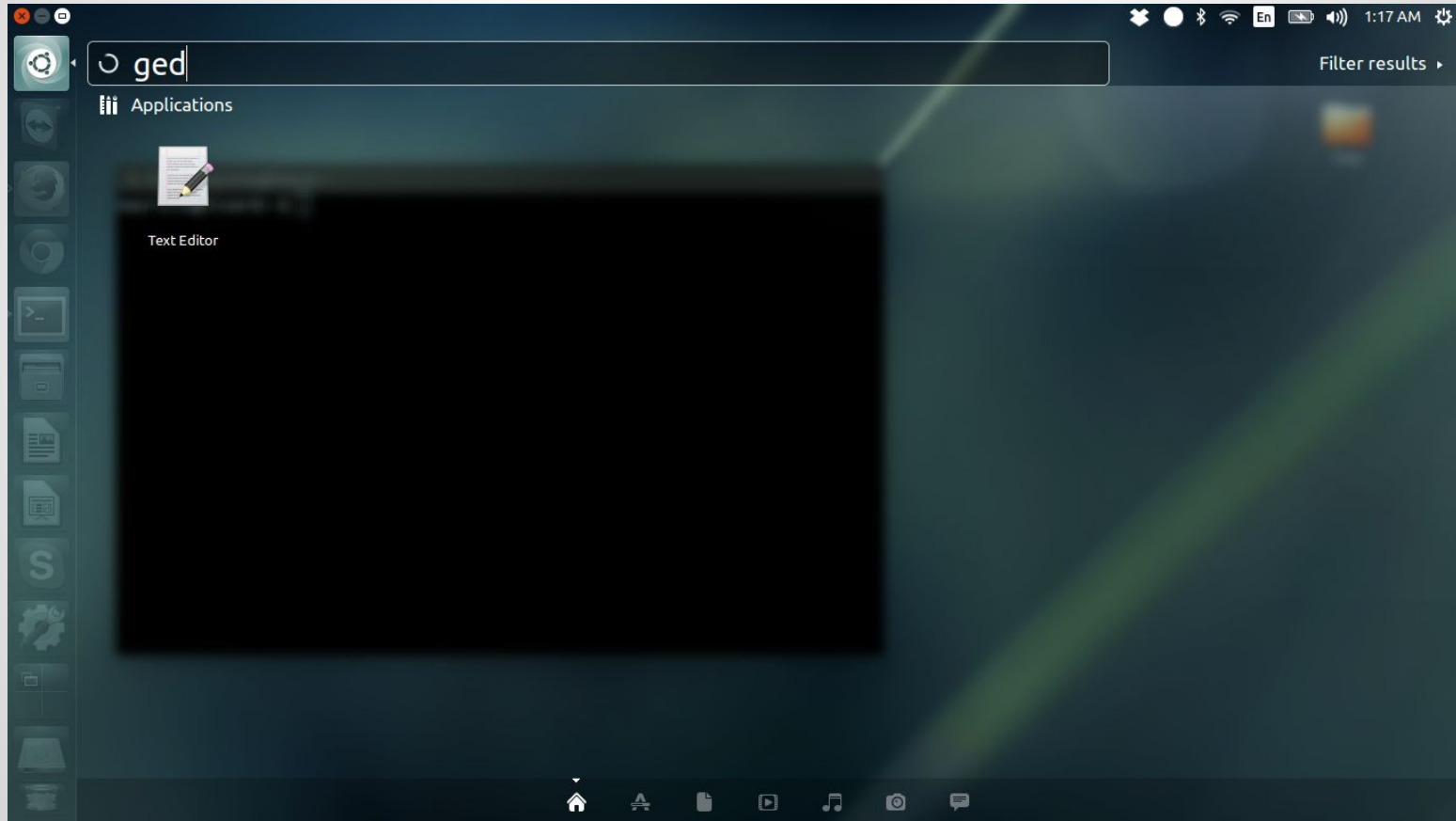
ENTRADA E SAÍDA DE DADOS E EXPRESSÕES ARITMÉTICAS



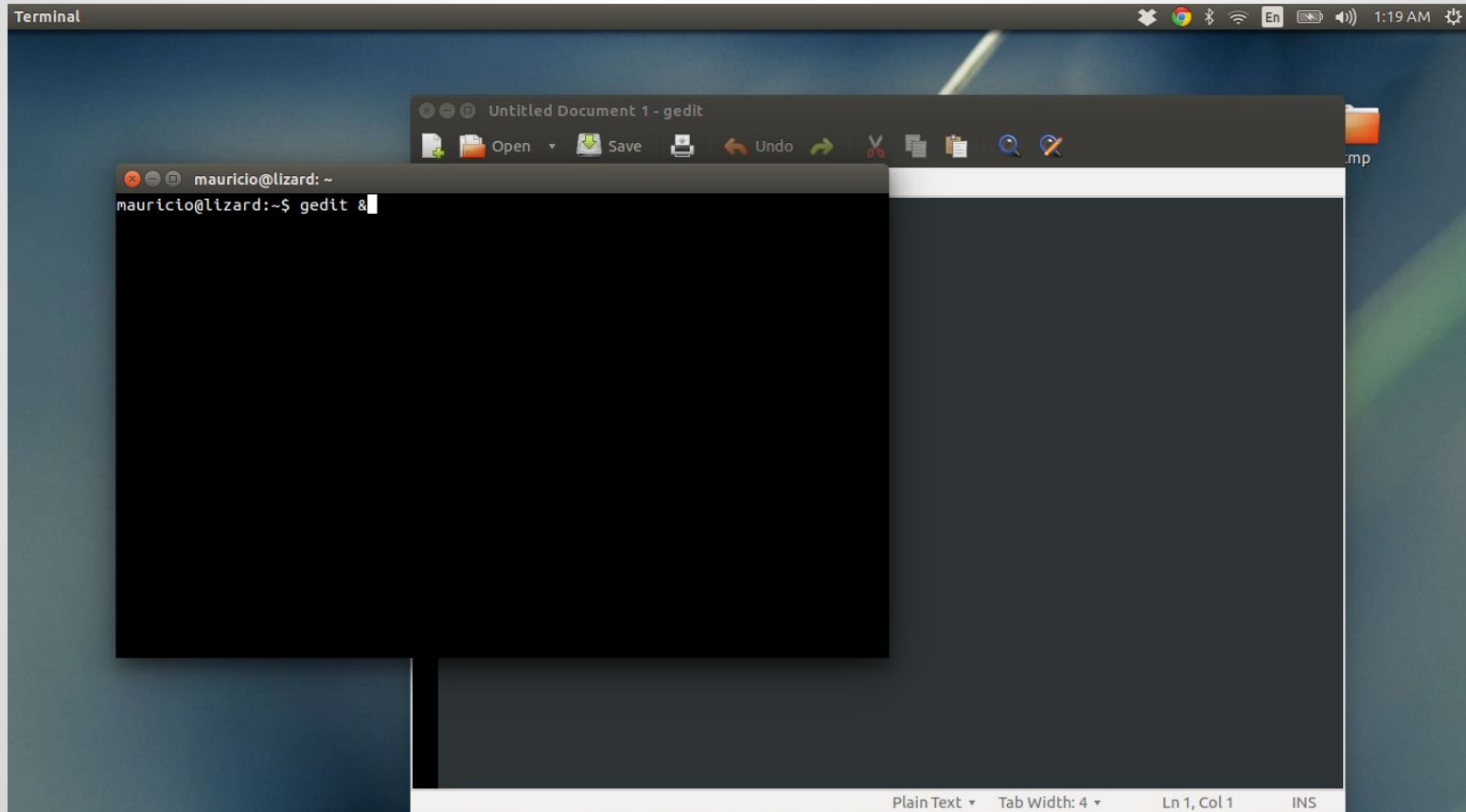
Abrindo um terminal



Abrindo um editor de texto



Abrindo um editor de texto



Comandos úteis em Linux

\$ ls

lista o conteúdo do diretório corrente

\$ ls nome_dir

lista o conteúdo do diretório nome_dir

\$ mkdir nome_dir

cria um diretório chamado nome_dir

\$ cd nome_dir

muda o diretório corrente para o diretório nome_dir

\$ cat nome_arq

imprime o conteúdo do arquivo nome_arq na tela

Escopo básico de C++

```
#include <iostream>
using namespace std;

int main(){
    //codigo
}
```

The diagram illustrates the basic structure of a C++ program. It consists of three main parts, each with an arrow pointing to a descriptive label:

- Cabeçalho** (Header): This label points to the preprocessor directives `#include <iostream>` and `using namespace std;`.
- Função principal** (Main function): This label points to the `int main(){` line.
- Parte que iremos alterar** (Part we will modify): This label points to the `//codigo` line.
- Fim do programa** (End of program): This label points to the closing brace `}`.

Declaração de variáveis

Especificação dos tipos de variáveis seguidos pelos nomes das variáveis (separados por vírgulas).

tipo var1, var2, var3, ..., varN;

Ex:

int a, b;

float pi = 3.1416;

Palavras reservadas

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Tipos de variáveis

int - inteiro [-2147483647 a 2147483647]

int a = 127;

long long - inteiro[-9223372036854775807 a 9223372036854775807]

long long a = 8000000000;

float - real de 32 bits

float x = 3.25;

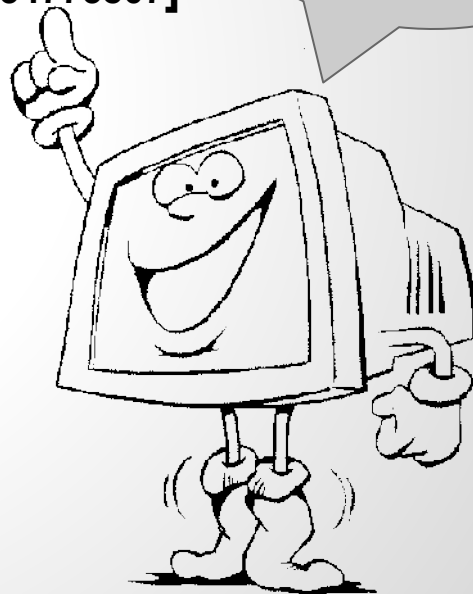
double - real de 64 bits

double y = 3.25364758697014;

char - caracter

char letra = 'a';

Os tipos de
variáveis sempre
devem ser escritos
em minúsculo!



Alo mundo!

Código:

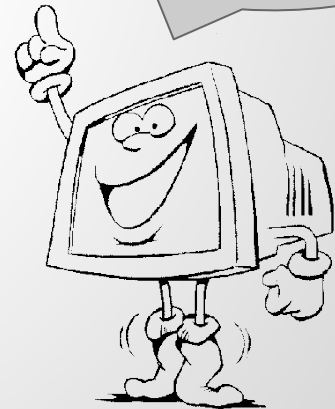
```
#include <iostream>
using namespace std;

int main(){
    cout << "Alo mundo!" << endl;
}
```

Escrever o código no editor.

Salvar como:
alo.cpp

Sempre usar a
extensão **.cpp** na
hora de salvar os
códigos!



Compilação e execução do programa

Para compilar (no terminal):

`g++ alo.cpp`

Para executar (no terminal):

`./a.out`



Saída padrão - cout

O **cout** recebe uma lista de parâmetros que serão impressos no terminal.

cout << par1 << par2 << ... << parN;

Os parâmetros podem ser variáveis ou constantes.

Ex:

```
int num = 24;
```

```
cout << "Resultado = " << num << endl;
```

Palavras e caracteres constantes devem ser postos entre aspas duplas!

endl = quebra de linha (ou seja, é como apertar enter num texto!).

Saída padrão - cout

Setar o número de casas decimais para valores reais com `setprecision` requer que `#include <iomanip>` seja adicionado ao cabeçalho. Ex:

```
float x = 3.1415;
```

```
cout << fixed << setprecision(2) << x << endl;
```

Este comando imprimirá o valor de `x` com 2 casas decimais seguido por uma quebra de linha. O efeito de `fixed/setprecision` é permanente. Caso seja preciso imprimir outros valores com outra quantidade de casas decimais, é preciso fazer o ajuste novamente.

Caracteres de escape

<i>Caractere</i>	<i>Significado</i>
<code>\a</code>	Aviso sonoro
<code>\b</code>	Retrocesso
<code>\f</code>	Salta uma página
<code>\n</code>	Nova linha
<code>\r</code>	Retorno do carro
<code>\t</code>	Tabulação horizontal
<code>\v</code>	Tabulação vertical
<code>\\</code>	Barra invertida
<code>\'</code>	Apóstrofe
<code>\"</code>	Aspas duplas
<code>\?</code>	interrogação
<code>\nnn</code>	Valor ASCII em octal
<code>\xnnn</code>	Valor ASCII em hexadecimal

Papagaio!

```
#include <iostream>
using namespace std;
```

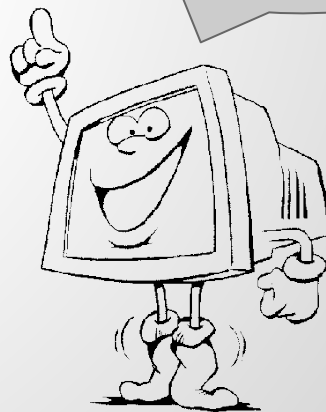
```
int main(){
    int x;
    cout << " Digite um número inteiro:" << endl;
    cin >> x;
    cout << " Inteiro digitado: " << x << endl;
}
```

Escrever o código no editor.

Salvar como:

papagaio.cpp

Sempre usar a
extensão **.cpp** na
hora de salvar os
códigos!



Papagaio!

Para compilar (no terminal):

`g++ papagaio.cpp`

Para executar (no terminal):

`./a.out`



Entrada padrão - cin

O ***cin*** recebe uma lista de parâmetros que serão digitados pelo teclado no terminal.

cin >> par1 >> par2 >> ... >> parN;

Ele identifica o tipo dos parâmetros de acordo com as declarações de variáveis.

Ex:

int a;

float b;

cin >> a >> b;

Expressões aritméticas

Atribuição: Quando atribuímos algo a uma variável, isto é, um valor, um character, etc (pode ser feita na declaração)

Ex:

```
int a, b, c, d, e = 5;
```

```
a = 10;
```

```
b = c = d = 0;
```

```
char b;
```

```
b = 'x';
```

Expressões aritméticas

Operador	Função
+	Soma
-	Subtração
*	Multiplicação
/	Divisão
%	Resto da divisão (inteiros)
()	Alteração de precedência

Ex:

$a = b * c + d;$

$x = (y - z) / \text{delta};$

Expressões aritméticas

Soma (+): $a + b = a$ mais b .

Ex:

```
int a, b, c, d;
```

```
a = 5;
```

```
b = 2 + 2; // b = 4
```

```
c = a + 3; // c = 8;
```

```
d = a + b + 1 + 2; // d = 12;
```

Expressões aritméticas

Subtração (-): $a - b = a$ menos b .

Ex:

```
int a, b, c, d;
```

```
a = 10;
```

```
b = 10 - a; // b = 0;
```

```
c = 1 - 2 - 3 - 4; // c = -8
```

```
d = b - c; // d = 8
```

Expressões aritméticas

Multiplicação (*): $a * b = a$ vezes b .

(Tem precedência sobre soma e subtração)

Ex:

```
int a, b, c, d;
```

```
a = 2;
```

```
b = 3 * 4; // b = 12
```

```
c = a * 2; // c = 4
```

Expressões aritméticas

Divisão (/): $a / b = a$ dividido por b .

(Tem precedência sobre soma e subtração)

Ex:

```
int a, b, c, d;
```

```
a = 5;
```

```
b = 10 / a; // b = 2;
```

```
c = a / 2; // c = 2;
```

```
d = 4 / 2 / 2; // d = (4 / 2) / 2; -> d = 2 / 2; -> d = 1;
```


Expressões aritméticas

Resto (%): $a \% b$ = resto (inteiro) da divisão de a por b.

Ex:

```
int a, b, c, d;
```

```
a = 2;
```

```
b = 16 % 10; // b = 6;
```

```
c = 84 % a; // c = 0;
```

```
d = a % b; // d = 2;
```

Expressões Aritméticas

Operador	Função
++	Incremento ($a++$ é o mesmo que $a = a + 1$)
--	Decremento ($a--$ é o mesmo que $a = a - 1$)
+=	Soma ($a += 2$ é o mesmo que $a = a + 2$)
-=	Subtração ($a -= 2$ é o mesmo que $a = a - 2$)
*=	Multiplicação ($a *= 2$ é o mesmo que $a = a * 2$)
/=	Divisão ($a /= 2$ é o mesmo que $a = a / 2$)

Ex:

`a++;`

`x /= delta+y/2;`

Resposta

Para conferir basta adicionar :

```
cout << x << endl;
```

DESVIOS CONDICIONAIS

Comandos de desvio

- Desvio simples

```
if ( <condição> )  
    comando;
```

comando único

Comandos de desvio

- **Desvio simples**

```
if ( <condição> ) {  
    comando1;  
    ...  
    comandoN;  
}
```

sequência de comandos

Média aritmética!

- media.cpp
- Descrição
 - Seu programa deve calcular a média aritmética de quatro notas e informar se o aluno foi aprovado. Alunos com média menor do que 7.0 estão reprovados.
- Entrada
 - Uma linha contendo quatro números reais.
- Saída
 - Seu programa deve imprimir a sentença “Aluno aprovado! Parabens!” seguida de uma quebra de linha caso o aluno seja aprovado.

Média aritmética!

Código:

```
#include <iostream>
using namespace std;

int main() {
    float a,b,c,d,media;
    cin >> a >> b >> c >> d;
    media = (a+b+c+d)/4.0;
    if(media >= 7.0) {
        cout << "Aluno aprovado! Parabens!\n";
    }
    return 0;
}
```

Para compilar:

```
$ g++ media.cpp
```

Para executar:

```
$ ./a.out
```


Comandos de desvio

- **Desvio composto**

```
if ( <condição> ) {  
    comando1;  
    ...  
    comandoN;  
}  
else {  
    comando1;  
    ...  
    comandoN;  
}
```

sequência de comandos

sequência de comandos

Média aritmética!

- media.cpp
- Descrição
 - Seu programa deve calcular a média aritmética de quatro notas e informar se o aluno foi aprovado. Alunos com média menor do que 7.0 estão reprovados.
- Entrada
 - Uma linha contendo quatro números reais.
- Saída
 - Seu programa deve imprimir a sentença “Aluno aprovado! Parabens!” ou “Aluno reprovado! Estude mais!” seguida de uma quebra de linha.

Média aritmética!

Código

```
#include <iostream>
using namespace std;

int main() {
    float a,b,c,d,media;
    cin >> a >> b >> c >> d;
    media = (a+b+c+d)/4.0;
    if(media >= 7.0) {
        cout << "Aluno aprovado! Parabens!\n";
    }
    else {
        cout << "Aluno reprovado! Estude mais!\n";
    }
    return 0;
}
```

Para compilar:

```
$ g++ media.cpp
```

Para executar:

```
$ ./a.out
```

Comandos de desvio

- **Desvio encadeado**

```
if ( <condição 1> ) {  
    if ( <condição 2> ) {  
        if ( <condição 3> ) {  
            if ( <condição 4> ) {  
                comandos;  
            }  
        }  
    }  
}
```

Condições

- Operador relacional

Operador	Ação
<	é menor que
<=	é menor que ou igual
>	é maior que
>=	é maior que ou igual
==	é igual a
!=	é diferente de

Condições

- Operador lógico

Operador	Ação	Formato da expressão
&&	and (e - conjunção lógica)	p && q
	or (ou - disjunção lógica)	p q
!	not (não - negação)	!p

Condições

- Operadores

$A > B$ (A maior do que B)

- Condições compostas

$A > B \ \&\& \ A > C$ (A maior do que B $\ \&\& \$ A maior do que C)

- Negação

$! (A > B) \Rightarrow A \leq B$ (A não é maior do que B)

Condições

- **Condições compostas**

$A > B \ \&\& \ A > C \ || \ A > D$ (A maior que B **E** A maior que C) **OU** A maior que D

$A > B \ \&\& \ (A > C \ || \ A > D)$ A maior que B **E** (A maior que C **OU** A maior que D)

- **Precedência**

- **&& (E)** equivale à multiplicação
- **|| (OU)** equivale à soma

Comandos de desvio

- Desvio múltiplo

```
if ( <variável> == valor1 ) {  
    comandos1;  
}else if ( <variável> == valor2 ) {  
    comandos2;  
}  
...  
else if ( <variável> == valorN ) {  
    comandosN;  
}else {  
    comandos_default;  
}
```

Comandos de desvio

- **Desvio múltiplo**

```
switch ( <variável> ) {  
    case valor1:  
        comandos1;  
        break;  
    case valor2:  
        comandos2;  
        break;  
    ...  
    case valorN:  
        comandosN;  
        break;  
    default:  
        comandos_default;  
}
```

ESTRUTURAS DE REPETIÇÃO

for - Repetição de contagem

```
for ( <inicialização> ; <condição> ; <incremento> )  
    comando;                *comando único*
```

for - Repetição de contagem

```
for ( <inicialização> ; <condição> ; <incremento> ) {  
    comando1;  
    ...  
    comandoN;  
}
```

sequência de comandos

Exemplo: programa que imprime a tabuada de 9

```
#include <iostream>
using namespace std;
int main() {
    for (int i=1; i <= 10; i++)
        cout << 9 << "x" << i << " = " << 9*i << endl;
    return 0;
}
```

while - Repetição pré-testada

```
while ( <condição> )
```

```
    comando;
```

comando único

while - Repetição pré-testada

```
while ( <condição> ) {  
    comando1;  
    ...  
    comandoN;  
}
```

sequência de comandos

Exemplo: programa que imprime a tabuada de 9

```
#include <iostream>
using namespace std;
int main() {
    int i = 1;
    while (i <= 10){
        cout << 9 << "x" << i << " = " << 9*i << endl;
        i++;
    }
    return 0;
}
```

break

Dentro de qualquer estrutura de repetição, é possível usar a instrução **break** para sair do loop

```
#include <iostream>
using namespace std;
int main() {
    int n;
    while (1) {
        cin >> n;
        if (n == 0)
            break;
        cout <<(n * 2) << endl;
    }
    cout <<"Fim";
    return 0;
}
```

// Sai do loop quando n == 0

continue

Dentro de qualquer estrutura de repetição, é possível usar a instrução **continue** para voltar para o início do loop

```
#include <iostream>
using namespace std;
int main() {
    for (int i=1; i<10; i++){
        if (i % 3 == 0)
            continue;
        cout << i << endl;
    }
    return 0;
}
```

// Não imprime os múltiplos de 3

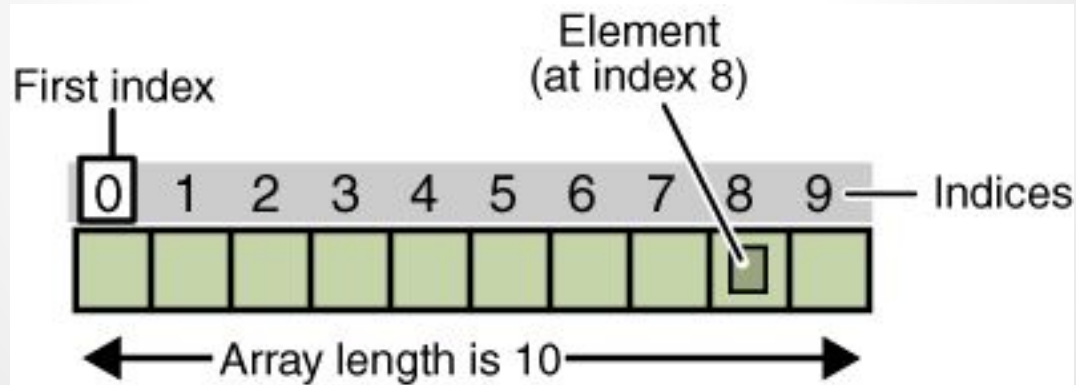


VETORES



Conceito

- Um vetor (array) é um conjunto de elementos consecutivos, todos do mesmo tipo, que podem ser acessados individualmente a partir de um único nome.



Como declarar um vetor?

```
#include <iostream>
using namespace std; // list -> std
int main() {
    float notas[100];
    int N = 5, tam = 27;
    int idade[N];
    double x[20*tam];
}
```

Acessando os valores do vetor

```
float notas[100];
```

```
cin >> notas[0] >> notas[1] >> ... >> notas[99];
```

Acessando os valores do vetor

```
float notas[100];
```

```
cin >> notas[0] >> notas[1] >> ... >> notas[99];
```


Acessando os valores do vetor

```
float notas[100];  
cin >> notas[0] >> notas[1] >> ... >> notas[99];  
int i;  
for(i = 0; i < 100; i++) {  
    cin >> notas[ i ];  
}
```

Acessando os valores do vetor

```
vetor[5] = 123;
```

```
indice = 3;
```

```
vetor[indice] = 100;
```

```
vetor[5*indice] = vetor[2]*2;
```

```
cout << vetor[indice+4*vetor[0]] << endl;
```

// Cuidado com valores fora do tamanho especificado

Carga inicial de vetores

```
int v[5] = {1, 2, 3, 4, 5};
```

// nesse caso, o compilador olha o tamanho automaticamente

```
int v[ ] = {1, 2, 3, 4, 5};
```

```
char str[6] = {'G', 'R', 'U', 'P', 'R', 'O'};
```

Par ou Ímpar

- **Descrição**

- Sua tarefa é, dada a sequência de **n números**, imprimir as posições ímpares e pares da mesma.

- **Entrada**

- A primeira linha contém um **inteiro n** ($1 \leq n \leq 1000$), representando a quantidade de elementos da sequência. A segunda linha contém **n inteiros** entre -10^6 e 10^6 .

- **Saída**

- A **primeira linha** deve conter os **números de posição ímpar** na sequência, separados por espaço, na ordem em que foram dados. A **segunda linha** deve conter os **números de posição par** na sequência, separados por espaço, na ordem em que foram dados.

Par ou Ímpar

```
#include <iostream>
using namespace std;
int main(){
    int a[1000];
    int i, n;
    // Lê a quantidade de elementos
    cin >> n;
    // Lê n elementos
    for(i = 0; i < n; i++) {
        cin >> a[i];
    }
```

```
// Imprime as posições ímpares
for(i = 0; i < n; i += 2) {
    if(i > 0) { cout << " "; }
    cout << a[i];
}
cout << endl;
// Imprime as posições pares
for(i = 1; i < n; i += 2) {
    if(i > 1) { cout << " "; }
    cout << a[i];
}
cout << endl;
return 0;
}
```

MATRIZES

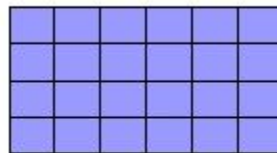
Matrizes

- Matrizes nada mais são do que vetores com múltiplas dimensões.

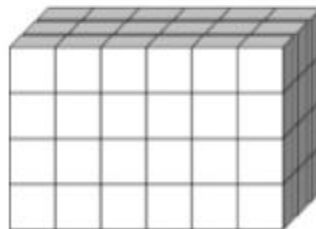
■ Vetor



■ Matriz 2D



■ Matriz 3D



Matrizes

- A declaração de uma matriz é semelhante a de um vetor, porém deve-se adicionar as novas dimensões:

float age[5];

age[0]	age[1]	age[2]	age[3]	age[4]

float arr[3][3];

arr	col[0]	col[1]	col [2]
row [0]	10	20	45
row [1]	42	79	81
row [2]	89	9	36

Matrizes

- Declaração e inicialização de uma matriz

```
int arr[3][3] = { {10, 20, 45} , {42, 79, 81} , {89, 9 , 36} };
```

arr	col[0]	col[1]	col [2]
row [0]	10	20	45
row [1]	42	79	81
row [2]	89	9	36

Matrizes

- No caso de uma matriz 2D, o primeiro tamanho corresponde à quantidade de linhas, e o segundo à quantidade de colunas.

```
float mat[3][4];  
mat[1][2] = 6;  
for (int i=0; i<3; i++)  
    for(int j=0; j<4; j++)  
        cin >> mat[i][j];
```

		colunas			
		0	1	2	3
linhas	0	4	5	3,5	7
	1			6	
	2				

Me Add Aí

- Descrição
 - Calcule a soma de duas matrizes.
- Entrada
 - Na primeira linha o número de linhas e o de colunas das matrizes. Nas N próximas linhas os elementos da primeira matriz. Após essas N linhas, temos outras N linhas com os elementos da segunda matriz.
- Saída
 - N linhas contendo os elementos da matriz que resulta da soma das duas matrizes lidas.

Me Add Aí

```
#include <iostream>
using namespace std;
int main() {
    int N, M, i, j;
    cin >> N >> M;
    int A[N][M], B[N][M], C[N][M];
    // Lê a matriz A
    for (i=0; i < N; i++)
        for (j=0; j < M; j++)
            cin >> A[ i ][ j ];
```

```
    for (i=0; i < N; i++) // Lê a matriz B
        for (j=0; j < M; j++)
            cin >> B[ i ][ j ];
```

```
    for (i=0; i < N; i++) // Calcula a matriz C
        for (j=0; j < M; j++)
            C[ i ][ j ] = A[ i ][ j ] + B[ i ][ j ];
```

```
    for (i=0; i < N; i++) { // Imprime a matriz C
        for (j=0; j < M-1; j++)
            cout << C[ i ][ j ] << " ";
        cout << C[ i ][ j ] << endl;
    }
    return 0;
}
```

STRINGS

Conceito

- Uma string é uma sequência de caracteres.

Index	0	1	2	3	4
Variable	H	e	l	l	o

```
string Variable = "Hello";
```

Codificação de Strings

- 9 e '9' possui valores diferentes:
 - **9** é o valor numérico do tab ('\t'), enquanto '9' tem valor numérico 57;
- É possível utilizar esses valores:
 - '0' + 1 == '1';
 - 'a' + 1 == 'b';
 - 'T' + 1 == 'U';

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Como declarar uma string?

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    string str = "Alô mundo!";
```

```
}
```

Como ler uma string?

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    string str;
```

```
    cin >> str; // lê uma palavra (para a leitura em espaço, tab ou enter)
```

```
                // ' ', '\t', '\n' não farão parte do conteúdo de str
```

```
}
```

Como ler uma string?

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    string str;
```

```
    getline(cin, str); // lê uma linha inteira (até o enter)
```

```
                        // '\n' não fará parte do conteúdo de str
```

```
}
```

Acessando a string

// Imprimir a n-ésima letra de uma string

```
cout << str[n] << endl;
```

// Descobrir o tamanho de uma string

```
int tam = str.size();
```

// Imprimindo caractere a caractere de uma string

```
for (i = 0; i < tam; i++)
```

```
    cout << str[i];
```

```
cout << endl;
```

Manipulando strings

```
string str1 = “Ola Mundo”,    str2, str3 = “Ola”;
```

```
str2 = str1; // copia str1 para str2
```

```
str2 = str2 + '!'; // adiciona o caractere '!' ao fim de str2
```

```
str3 = str3 + “ Mundo”; // adiciona a string “ Mundo” ao fim de str3
```

```
if (str1 == str2) // compara str1 e str2 (<, <=, >, >= e != também existem)
```

```
    cout << “As strings são iguais” << endl;
```

```
else
```

```
    cout << “As strings são diferentes” << endl;
```

Manipulando strings

Nome	Função
strcpy(s1,s2)	Copia s2 em s1.
strcat(s1,s2)	Concatena s2 ao final de s1.
strlen(s1)	Retorna o tamanho de s1.
strcmp(s1,s2)	Retorna 0 se (s1 == s2), Menor que 0 se (s1 < s2), Maior que 0 se (s1 > 2)
strchr(s1,ch)	Retorna um ponteiro para a primeira ocorrência de ch em s1
strstr(s1,s2)	Retorna um ponteiro para a primeira ocorrência de s2 em s1

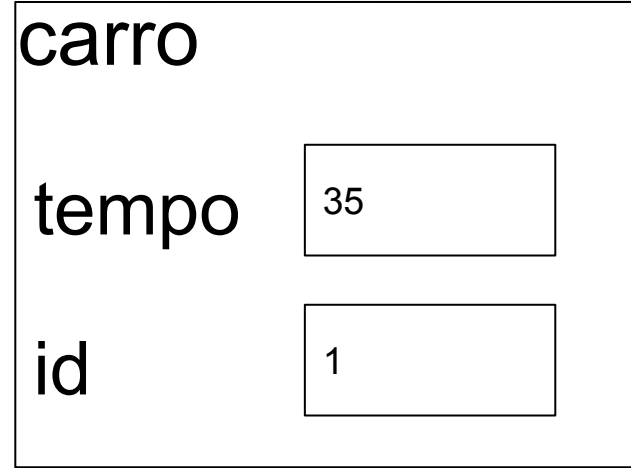
Structs (Estruturas)

Declaração de Struct

```
struct nome_da_struct{  
    tipo1 campo1;  
    tipo2 campo2;  
    ...  
    tipon campon;  
};
```


Exemplo de struct

```
struct carro{  
    int tempo;  
    int id;  
};
```



Declaração de variável do tipo struct

```
carro c1;
```

```
carro c2={ 10 , 2 };
```

Atribuição de valores

Os campos individuais são referenciados por meio do operador (.)

```
carro c1, c2, c3;
```

```
cin >> c1.tempo >> c1.id;
```

```
c2.tempo = 30;
```

```
c2.id = 2;
```

```
c3 = c1;
```

Exemplo de struct

```
#include <iostream>
using namespace std;

struct carro{
int tempo;
int id;
};
```

```
int main() {
    carro c1, c2;
    cin >> c1.tempo >> c1.id;
    c2.tempo = 30;
    c2.id = 2;
    cout << "carro1: ID =" << c1.id ;
    cout<< " tempo =" << c1.tempo << endl;

    cout << "carro2: ID =" << c2.id ;
    cout<< " tempo =" << c2.tempo << endl;
```

Vetor de struct

```
int n =2;
```

```
carro carros[n];
```

```
for (int i=0;i<n;i++)
```

```
    cin >> carros[i].tempo >> carros[i].id;
```

```
for (int i=0;i<n;i++)
```

```
    cout << "carro : " << carros[i].id << " tempo :" << carros[i].tempo << endl;
```

Vetor de struct

```
#include <iostream>
using namespace std;
```

```
struct carro{
int tempo;
int id;
};
```

```
int main() {
    int n =2;
    carro carros[n];
    for (int i=0;i<n;i++)
        cin >> carros[i].tempo >> carros[i].id;

    for (int i=0;i<n;i++)
        cout << "carro : " << carros[i].id ;
        cout<< " tempo : " << carros[i].tempo;
        cout << endl;
```

FUNÇÃO

O que é uma função?

- **Funções** são blocos de código que podem ser **chamados** em outros pontos do código
 - Chamar uma função significa pedir executar o seu bloco de código

Declaração de função

tipo_de_retorno **nome_funcao**(tipo par1, tipo par2, ...)

Exemplo

```
int mult2(int a, int b) {  
    return a*b;  
}
```

Chamada da função

```
int mult2(int a, int b) {  
    return a * b;  
}  
  
int main() {  
    int a = 3, b = 5, c;  
    c = mult2(a, b)  
    cout << mult2(10, a) << endl;  
}
```

FUNÇÃO RECURSIVA

Recursão

A recursão resolve um problema complexo a partir de uma versão menor do problema.

Exemplo: Fatorial de um número N

$$N! = N * (N-1) * (N-2) * \dots * 3 * 2 * 1$$

$$N! = N * (N-1)!$$

$$(N-1)! = (N-1) * (N-2)!$$

Recursão

Exemplos de números fatoriais

$$0! = 1$$

$$1! = 1$$

$$2! = 2 * 1$$

$$3! = 3 * 2 * 1$$

$$4! = 4 * 3 * 2 * 1$$

Recursão

Exemplos de números fatoriais

$$0! = 1$$

$$1! = 1$$

$$2! = 2 * 1 \quad \longleftrightarrow \quad 2! = 2 * 1!$$

$$3! = 3 * 2 * 1 \quad \longleftrightarrow \quad 3! = 3 * 2!$$

$$4! = 4 * 3 * 2 * 1 \quad \longleftrightarrow \quad 4! = 4 * 3!$$

Recursão

Exemplo: Fatorial de um número N

$$N! = \begin{cases} N * (N - 1)! & , \text{ se } N > 1 \\ 1 & , \text{ se } N = 1 \text{ ou } 0 \end{cases}$$

Recursão

Exemplo: Fatorial de um número N

$$N! = \begin{cases} N * (N - 1)! & , \text{ se } N > 1 \\ 1 & , \text{ se } N = 1 \text{ ou } N = 0 \end{cases}$$


Condição de parada

Função recursiva

Uma função é recursiva quando a função chama a si mesma.

Exemplo $N!$ (fatorial de N)

```
int fatorial (int N) {  
    if (N==0) || (N ==1)  
        return (1);  
    return (N * fatorial(N-1));  
}
```



BIBLIOTECA VECTOR



Vetores - vector

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std; // vector -> std
```

```
int main() {
```

```
    int v1[100];           // vetor tradicional com 100 elementos
```

```
    vector<int> v2(100);    // vector com 100 elementos pré-alocados
```

```
    for(int i=0; i < 100; i++) {
```

```
        cin >> v1[i] >> v2[i];
```

```
    }
```

```
}
```

// por que usar vector?

Vetores - push_back

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v;
    for(int i=0; i < 1000; i++) {
        int j;
        cin >> j;
        v.push_back(j); // função push_back adiciona elemento no final do vetor
                        // e aloca mais espaço caso necessário
    }
}
```

Vetores - size

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v;
    int x;
    while(cin >> x && x >= 0)
        v.push_back(x);
    for(int i=0; i < v.size(); i++) // função size retorna a qtdade de elementos do vetor
        cout << v[i] << endl; // o acesso ao vetor é realizado normalmente
}
```

Vetores - clear

...

```
int main() {  
    vector<int> v;  
    int x;  
    while(cin >> x && x >= 0)  
        v.push_back(x);  
    for(int i=0; i < v.size(); i++)  
        cout << v[i] << endl;  
    v.clear(); // função clear remove todos os elementos do vetor  
    while(cin >> x && x >= 0)  
        v.push_back(x);  
}
```

Vetores - iterator

```
#include <iterator>
```

```
...
```

```
int main() {  
    vector<int> v;
```

```
...
```

```
// iterator é um ponteiro para elementos
```

```
for(vector<int>::iterator it = v.begin();    it != v.end();    it++)  
    cout << *it << endl; // não imprima o iterator, e sim o valor apontado por ele!  
}
```

Vetores - erase

...

```
int main() {  
    vector<int> v;  
    int x;  
    while(cin >> x && x >= 0)  
        v.push_back(x);  
    v.erase(v.begin());           // apaga o primeiro elemento  
    v.erase(v.begin()+1);        // apaga o segundo elemento  
    v.erase(v.begin()+2, v.end()); // apaga todos os elementos exceto os dois primeiros  
}
```


Vetores - vector

Saiba mais em:

<http://www.cplusplus.com/reference/vector/vector/>

Métodos de ordenação

- Consiste em organizar uma sequência de elementos de modo que estabeleçam alguma relação de ordem.

$$a_1 \leq a_2 \leq a_3 \leq \dots \leq a_n$$

- O objetivo de ordenação é facilitar buscas, comparações e operações em conjuntos de dados.
- Exemplos reais:
 - Lista de alunos por nota
 - Tabela de produtos por preço
 - Banco de dados de clientes por ID



ORDENAÇÃO



Métodos de ordenação

- Em C++ , a biblioteca padrão **algorithm** fornece uma função predefinida e pronta para uso `sort()` para realizar a operação de ordenação.
 - 1. Ordenação em ordem crescente
 - `sort()`
 - 2. Ordenação em ordem decrescente

Ordenação por Seleção/ SELECTIONSORT

- Selection Sort é um algoritmo de ordenação simples.
- A ideia:
 - Selecionar repetidamente o menor (ou maior) elemento da lista e colocá-lo na posição correta.
 - Funciona por comparação.

Ordenação por Seleção/ Selectionsort

```
void selecao (int *vetor, int total) {  
    int i, j, posMin, valorMin, troca;  
  
    for (i= 0; i < total-1; i++) {  
        troca= 0;  
        posMin= i;  
        valorMin= vetor[i];  
        for (j= i+1; j < total; j++) {  
            if (vetor[j] < valorMin) {  
                posMin= j;  
                valorMin= vetor[j];  
                troca= 1;  
            }  
        }  
        if (troca) {  
            vetor[posMin]= vetor[i];  
            vetor[i]= valorMin;  
        }  
    }  
}
```

5 3 4 1 2

Selection Sort

Ordenação - sort

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;           // sort -> std
```

```
int main() {
```

```
    vector<int> v;
```

```
    for(int i=0; i < 1000; i++) {
```

```
        int j; cin >> j;
```

```
        v.push_back(j);
```

```
    }
```

```
// ordenação com complexidade ~ NlogN (não estável)
```

```
sort (v.begin(), v.end());
```

```
}
```

Saída:

5 2 7 1 7

1 2 5 7 7

Ordenação - stable_sort

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std; // sort -> std
int main() {
    vector<int> v;
    for(int i=0; i < 1000; i++) {
        int j; cin >> j;
        v.push_back(j);
    }
    stable_sort (v.begin(), v.end()); // ordenação com complexidade = NlogN
}
```

Saída:

5 2 7 1 7

1 2 5 7 7

Ordenação - reverse

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std; // sort -> std
int main() {
    vector<int> v;
    for(int i=0; i < 1000; i++) {
        int j; cin >> j;
        v.push_back(j);
    }
    reverse(v.begin(), v.end()); // ordenação em ordem decrescente
}
```

Saída:
5 2 7 1 7
7 7 5 2 1

Ordenação - sort & struct

```
struct pessoa {  
    int id;  
    string nome;  
};  
  
bool cmp(pessoa i, pessoa j) {  
    return ( i.id < j.id || i.id == j.id && i.nome < j.nome);  
}  
  
int main() {  
    vector<pessoa> v; //...  
    stable_sort (v.begin(), v.end(), cmp);  
}
```

Ordenação - sort & struct

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>

using namespace std;

struct pessoa {
    int id;
    string nome;
};

bool cmp(pessoa i, pessoa j) {
    return ( i.id < j.id || i.id == j.id && i.nome < j.nome);
}

// continua ...
```

```
int main() {
    vector<pessoa> v;
    pessoa x;

    for (int i=0; i<100 ; i++){
        cin >> x.id >> x.nome;
        v.push_back(x);
    }
    // vetor sem ordenação
    for (int i=0; i<100 ; i++)
        cout << v[i].id << " " << v[i].nome << endl;
    cout << endl;

    stable_sort (v.begin(), v.end(), cmp);

    cout << "Vetor ordenado" << endl;
    for (int i=0; i<100 ; i++) // vetor ordenado
        cout << v[i].id << " " << v[i].nome << endl;
}
```

Saída:

```
5 Jose
3 Maria
6 Carlos
6 Ana
Vetor ordenado
3 Maria
5 Jose
6 Ana
6 Carlos
```

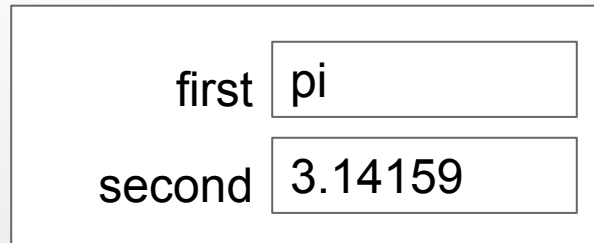


PARES DE DADOS



Pares de dados - pair

- Estrutura para armazenar pares de valores de tipos arbitrários, que são tratados como se fossem um único valor.
- Os elementos de um par são armazenados em duas variáveis-membro:
 - first - primeiro elemento do par
 - second - segundo elemento do par
 - Exemplo



Pares de dados - pair

```
#include <iostream>
```

```
#include <utility>
```

```
using namespace std; // pair -> std
```

```
int main() {
```

```
    pair<string,double> p; // tipos dos dados do par
```

```
    p.first = "pi"; // acessa primeira parte do par
```

```
    p.second = 3.14159; // acessa a segunda parte do par
```

```
    cout << p.first << " " << p.second << endl;
```

```
}
```

first	pi
second	3.14159

Saída: pi 3.14159

Pares de dados - inicialização de pair

```
#include <iostream>
```

```
#include <utility>
```

```
using namespace std; // pair -> std
```

```
int main() {
```

```
    pair<string,double> p1("pi", 3.14159); // tipos dos dados do par
```

```
    pair<string,double> p2 = { "abc", 1.111}; // tipos dos dados do par
```

```
    cout << p1.first << " " << p1.second << endl;
```

```
    cout << p2.first << " " << p2.second << endl;
```

```
}
```

Saída:

pi 3.14159

abc 1.111

Pares de dados - make_pair

```
#include <iostream>
#include <utility>
using namespace std; // pair -> std
int main() {
    pair<string,double> p1; // tipos dos dados do par
    string str = "exemplo";
    double num = 56.98;
    p1 = make_pair(str, num);
    auto p2 = make_pair(23, 22);    // declara um pair<int, int>
    cout<< p1.first << " " << p1.second << endl;
    cout<< p2.first << " " << p2.second << endl;
}
```

Saída:
exemplo 56.98
23 22

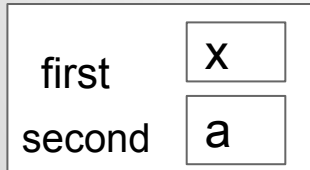
Pares de dados - swapping

```
#include <iostream>
#include <utility>
using namespace std; // pair -> std
int main() {
    pair<char, int>p1 = make_pair('A', 1);
    pair<char, int>p2 = make_pair('B', 2);
    p1.swap(p2);      // Realiza a troca os dados dos pares
    cout<< p1.first << " " << p1.second << endl;
    cout<< p2.first << " " << p2.second << endl;
}
```

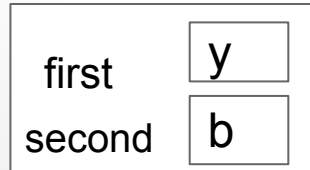
Saída:
B 2
A 1

Comparação de Pares de dados

Comparação	$\text{par1}(x, a) \text{ , } \text{par2}(y, b)$
$\text{par1} < \text{par2}$ $(x, y) < (a, b)$	SE E SOMENTE SE $x < y$ OU $(x == y \text{ E } a < b)$
$\text{par1} > \text{par2}$ $(x, y) > (a, b)$	$x > y$ OU $(x == y \text{ E } a > b)$



par1



par2

Comparação de Pares de dados

```
#include<iostream>
#include<utility>
using namespace std;
int main(){
    pair<int,int> P1 = {10,12};
    pair<int,int> P2 = {10,13};

    if (P1 > P2)
        cout << "P1 ganha" << endl;
    else if (P2 > P1)
        cout << "P2 ganha" << endl;
    else
        cout << "empate" << endl;
    return 0;
}
```

Saída:
P2 ganha

Sobrecarga de operadores - pair

// comparação entre pares utiliza a primeira parte e desempata pela segunda

```
bool operator<(pair<int,int> a, pair<int,int> b) {  
    return a.first < b.first || a.first == b.first && a.second < b.second;  
}
```

// você pode redefinir um operador de comparação:

```
bool operator<(pair<int,int> a, pair<int,int> b) {  
    return a.second < b.second || a.second == b.second && a.first < b.first;  
}
```

Comparação de Pares de dados - pair

```
#include <utility> // ...
```

```
// comparação entre pares utiliza a primeira parte e desempata pela segunda
```

```
int main(){  
    pair<char, int>p1 = make_pair('C', 1);  
    pair<char, int>p2 = make_pair('B', 2);  
  
    cout << "Pair 1 = " << p1.first << " " << p1.second << endl;  
    cout << "Pair 2 = " << p2.first << " " << p2.second << endl;  
    cout << (p1 < p2) << endl;  
}
```

Comparação de Pares de dados - pair

// Redefinindo o operador de comparação

// comparação entre pares utilizando a segunda parte e desempata pela primeira

```
bool operator<(pair<char, int> a, pair<char, int> b) {  
    return a.second < b.second || a.second == b.second && a.first < b.first;  
}
```

```
int main(){  
    pair<char, int>p1 = make_pair('C', 1);  
    pair<char, int>p2 = make_pair('B', 2);  
    cout << "Pair 1 = " << p1.first << " " << p1.second << endl;  
    cout << "Pair 2 = " << p2.first << " " << p2.second << endl;  
    cout << (p1 < p2) << endl;  
}
```

Vetor de Pares de dados

```
#include <utility> // ...

using namespace std; // pair -> std

int main() {
    int n = 3, n1, n2;
    vector < pair<int,int> > vet;           // vetor de pair <int, int>
    for (int i = 0; i<n; i++){
        cin >> n1 >> n2;
        vet.push_back(make_pair(n1, n2));
    }
    for (int i = 0; i<n; i++)
        cout<< vet[i].first << " " << vet[i].second << endl;
}
```

Saída:

4 5

7 8

3 9

Ordenação de Vetor de Pares de dados

```
#include <algorithm>
#include <utility> // ...

int main() {
    int n = 3, n1, n2;
    vector < pair<int,int> > vet;           // vetor de pair <int, int>
    for (int i = 0; i<n; i++){
        cin >> n1 >> n2;
        vet.push_back(make_pair(n1, n2));
    }
    stable_sort (vet.begin(), vet.end());
}
```

Saída:

3 4

6 4

8 2

Ordenação pela segunda parte de Pares de dados

```
#include <algorithm>
#include <utility> // ...

bool cmp(pair<int,int> a, pair<int,int> b) {
    return a.second < b.second || a.second == b.second && a.first < b.first;
}

int main() {
    int n, n1, n2;
    vector < pair<int,int> > vet;          // vetor de pair <int, int>
    for (int i = 0; i<n; i++){
        cin >> n1 >> n2;
        vet.push_back(make_pair(n1, n2));
    }
    stable_sort (vet.begin(), vet.end(), cmp);
}
```

Saída:

8 2

3 4

6 4

Ordenação pela segunda parte de Pares de dados

```
#include <iostream>
#include <algorithm>
#include<utility>
#include<vector>
using namespace std;

bool cmp(pair<char, int> a, pair<char, int> b) {
    return a.second < b.second || a.second
== b.second && a.first < b.first;
}

// continua ...
```

```
int main(){
    int n1, n2, n = 3;;
    vector < pair<int,int> > vet;
    for (int i = 0; i<n; i++){
        cin >> n1 >> n2;
        vet.push_back(make_pair(n1, n2));
    }
    stable_sort (vet.begin(), vet.end(), cmp);
    for (int i = 0; i<n; i++){
        cout << vet[i].first << " " << vet[i].second << endl;
    }
}
```

Saída:

8 2

3 4

6 4

Referências

[Joel Saade](#). C++ STL - Guia de Consulta Rápida. Novatec 2006

Pair

<http://www.cplusplus.com/reference/utility/pair/>

sort

<http://www.cplusplus.com/reference/algorithm/sort/>